Hochschule Konstanz
Fakultät Informatik

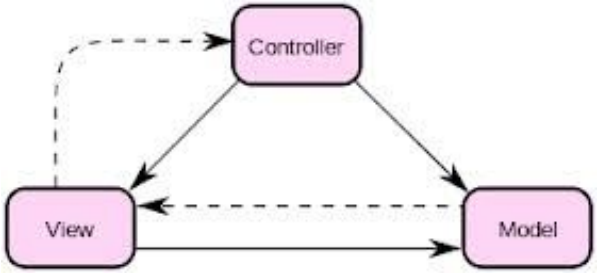
Software Architecture

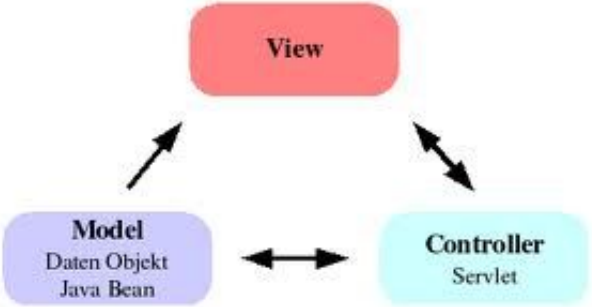
Slick

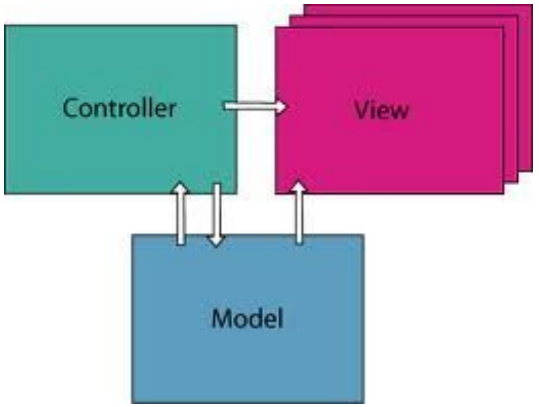
Prof. Dr. Marko Boger

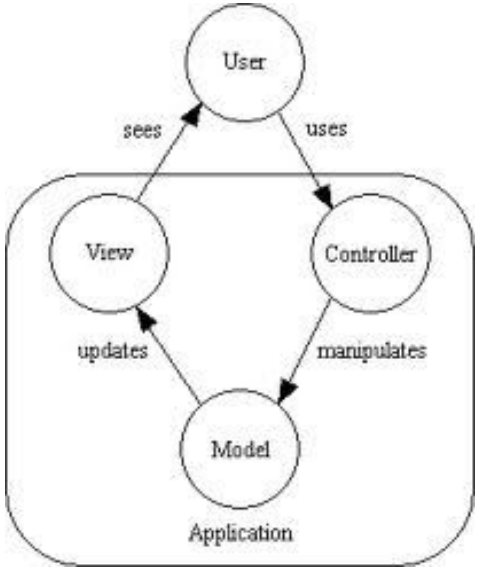
Model View Controller

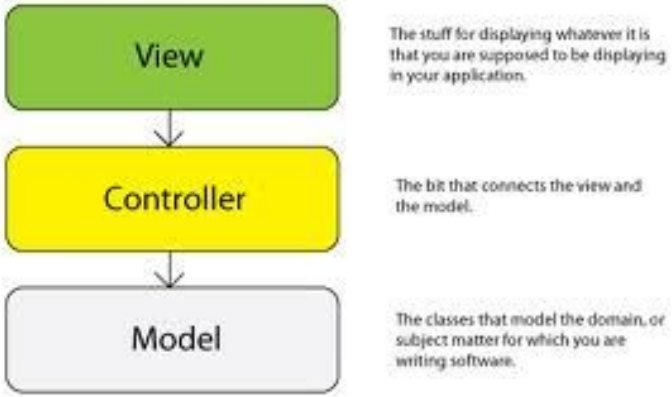
Most UI frameworks are MVC-based. But what does that mean?

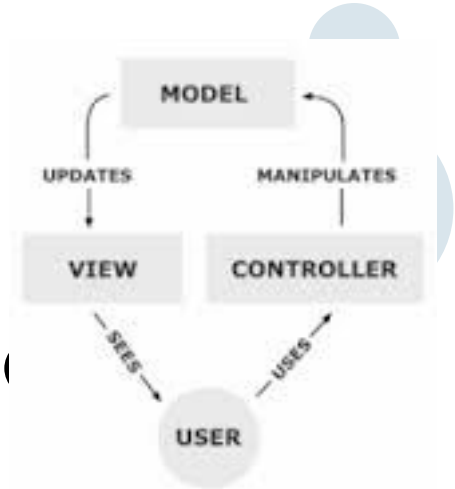






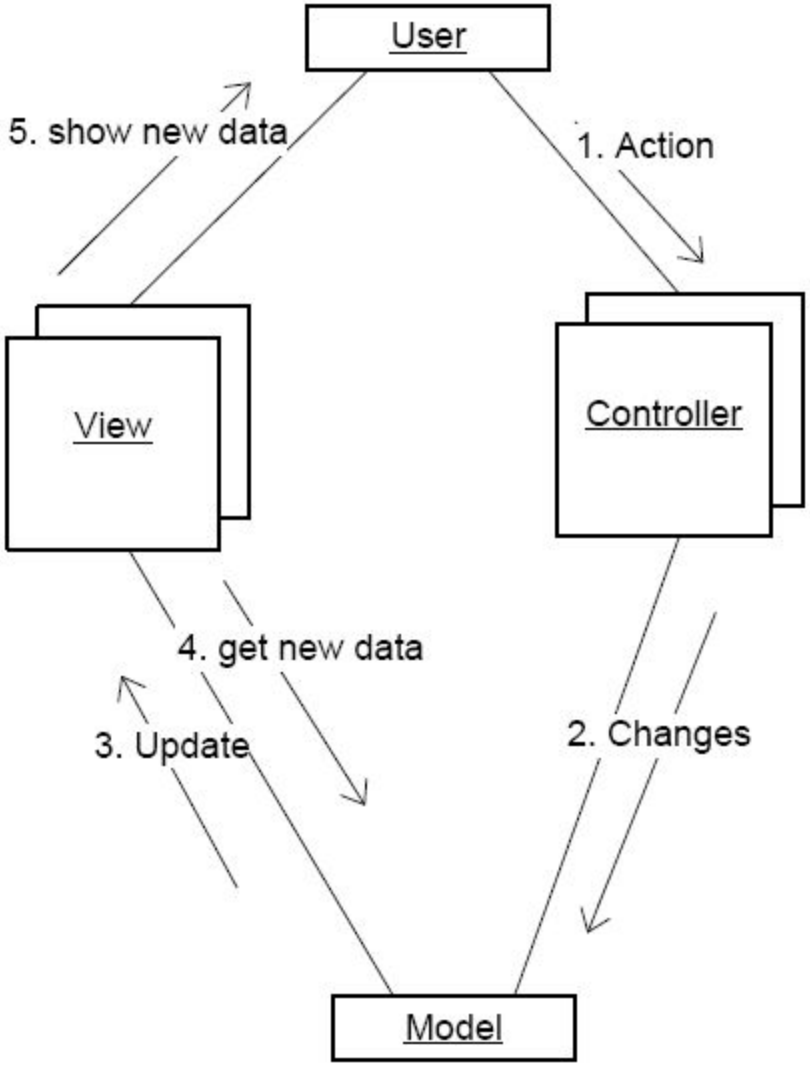






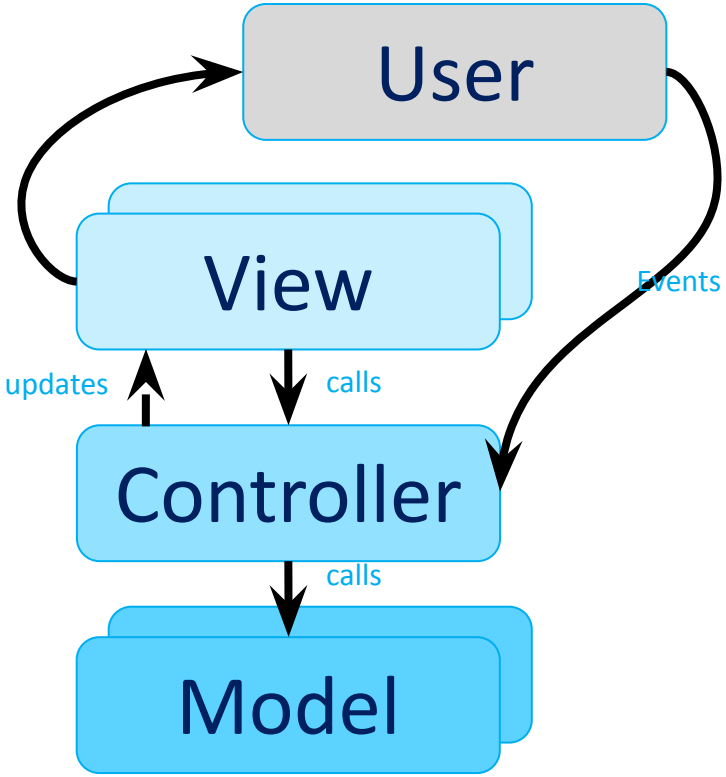
MVC

- Developed in the 1970's for Smalltalk
  - Reenskaug
- THE Standard for GUIs
- Key ideas
  - Separated Presentation
    - Several independant presentations
    - Potentially simultaneously
    - Including command line and GUI
  - Controllers handle input
  - Changes in state are communicated via Observer P.
  - Event driven



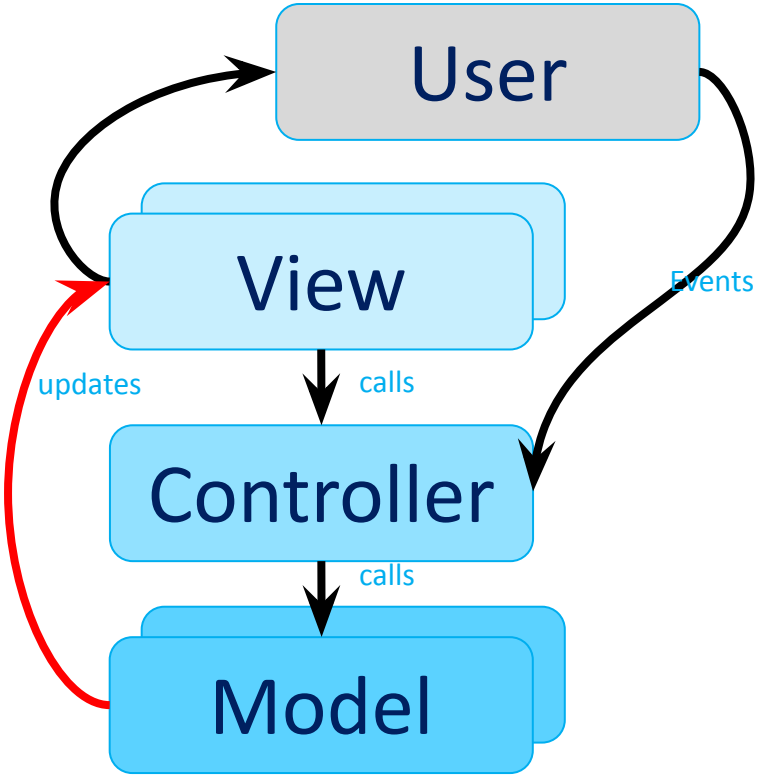
MVC with Layers

- The view is displayed to the user
- All events or actions from the user go directly into the controller
- The controller changes one or more model elements and updates the view
- The view can query the controller for new data, which will get the data from the model
- The updated view is displayed



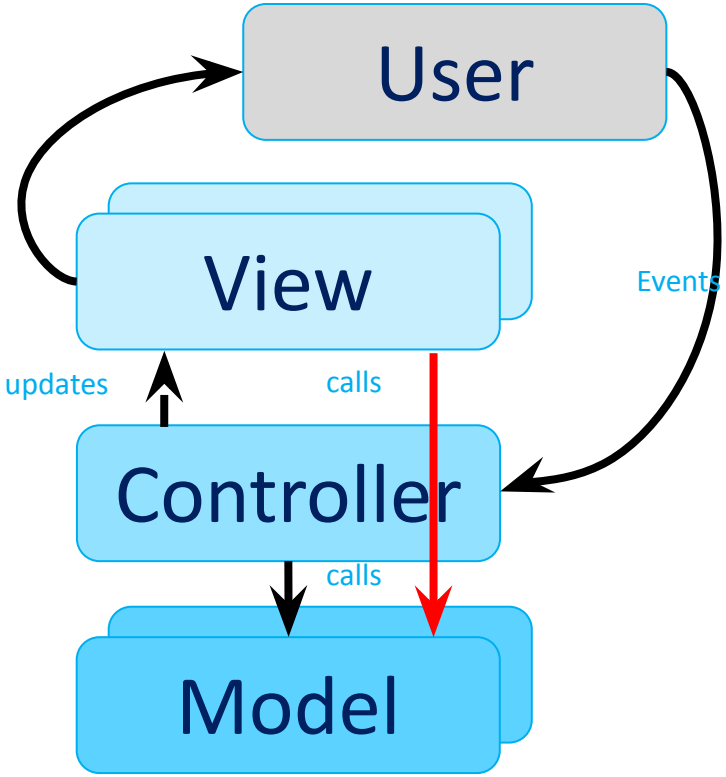
Non-strict Layering I

- The model can update the view directly
- Disadvantage:
  - The model does have an indirect connection to the view. Persisting the model as object tree pulls the view into the database with it.

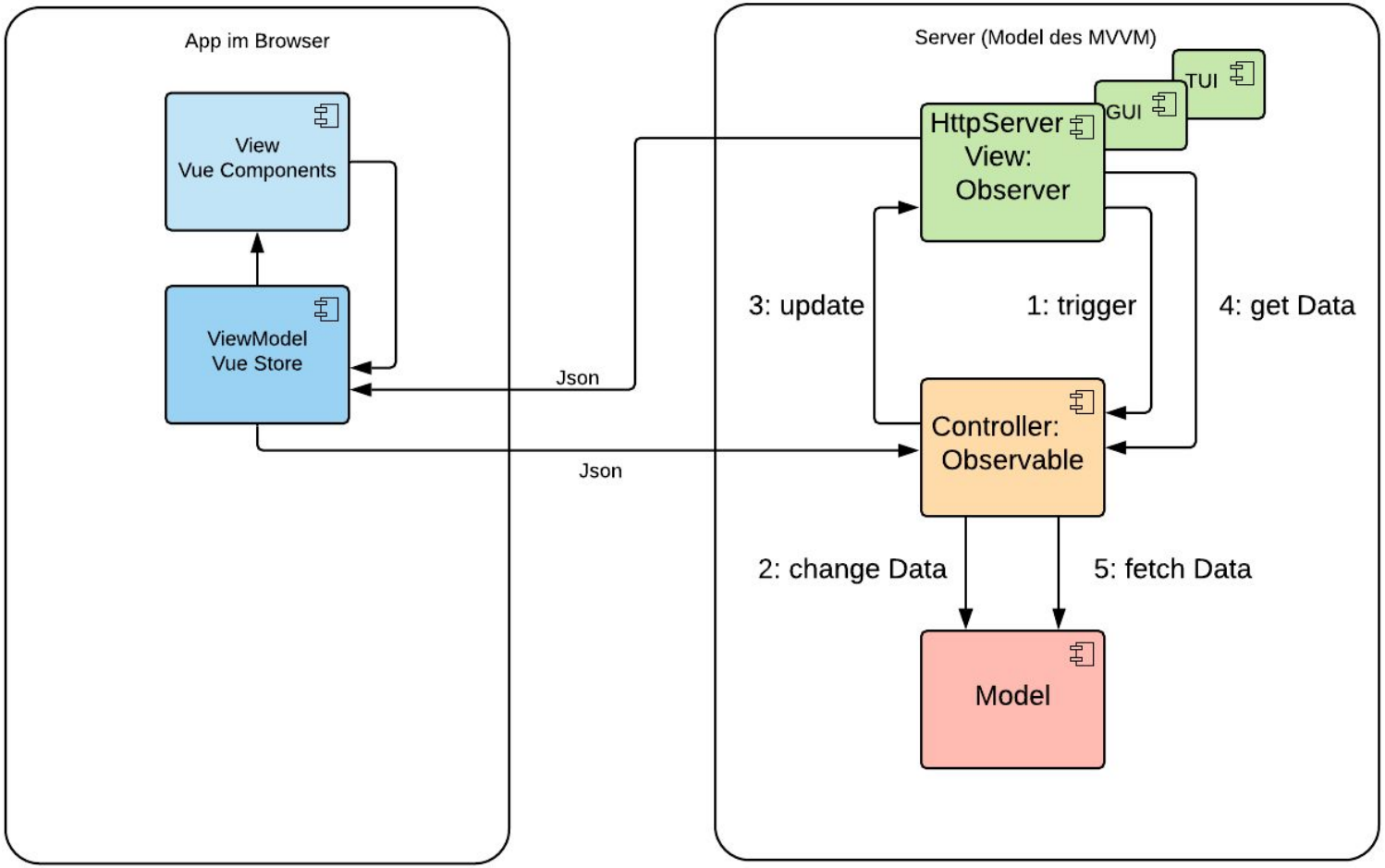


Non-strict Layering II

- The view can make calls for updating data directly to the model
- Disadvantages:
  - If the model is replaced with a new instance from a database, the view does not know about it.



General Architecture

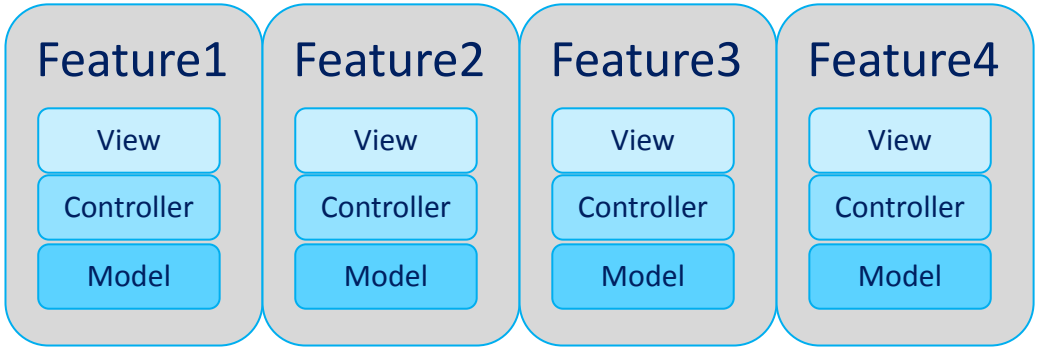


Architecture Styles

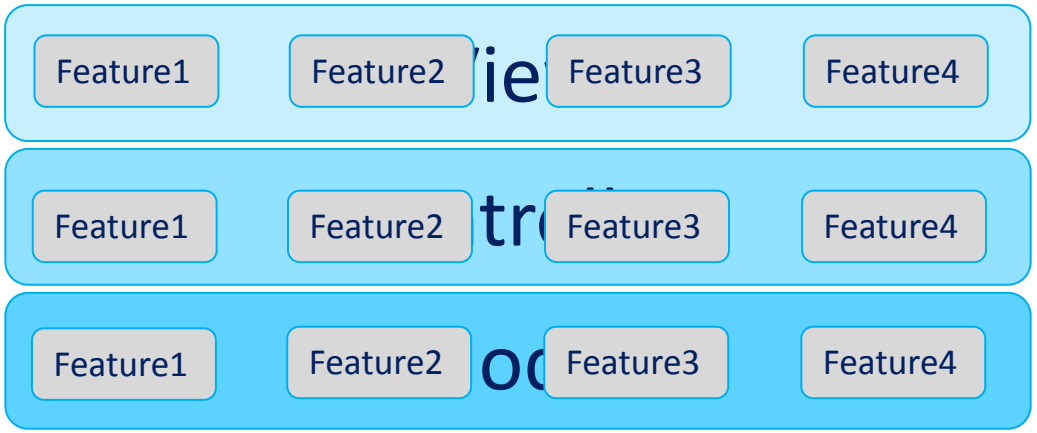
	Server	Client
Console	Model, Controller, View	
Fat Client		Model, Controller, View
Rich Client	Model	Controller, View
Web Application	Model, Controller	View
RIA	Model, Controller	Controller, View
Mobile	Model	Model, Controller, View

Cutting the Architecture by Feature or Layer

- Built by Feature

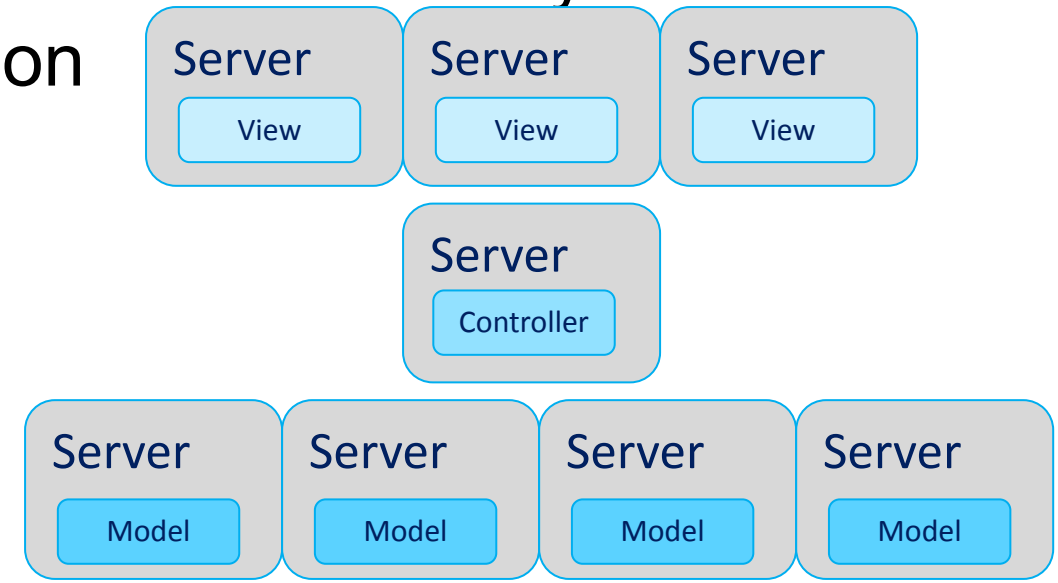


- Built by Layer

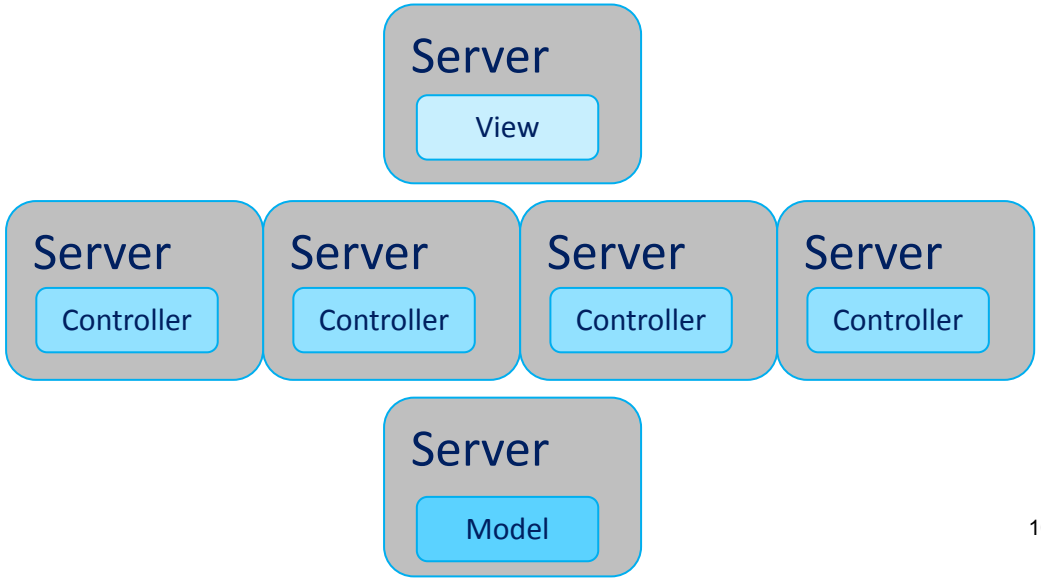


Cutting the Architecture by Distribution

- 3-tier with
  - Replicated View
  - Single Controller
  - Replicated Model



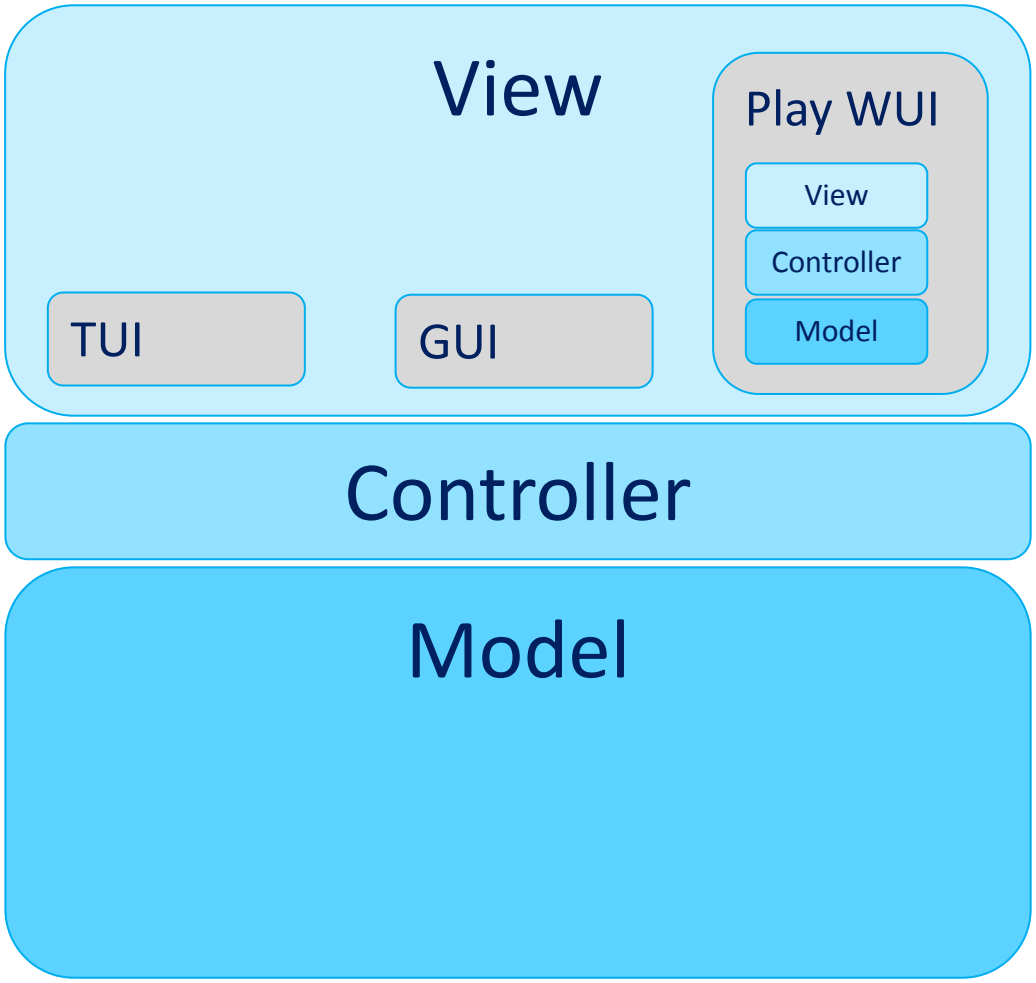
- 3-tier with
  - Single View
  - Replicated Controller
  - Single Model



Architecture for this Lecture

- 3 different views
- Play has its own internal MVC

- 3 strickt layers

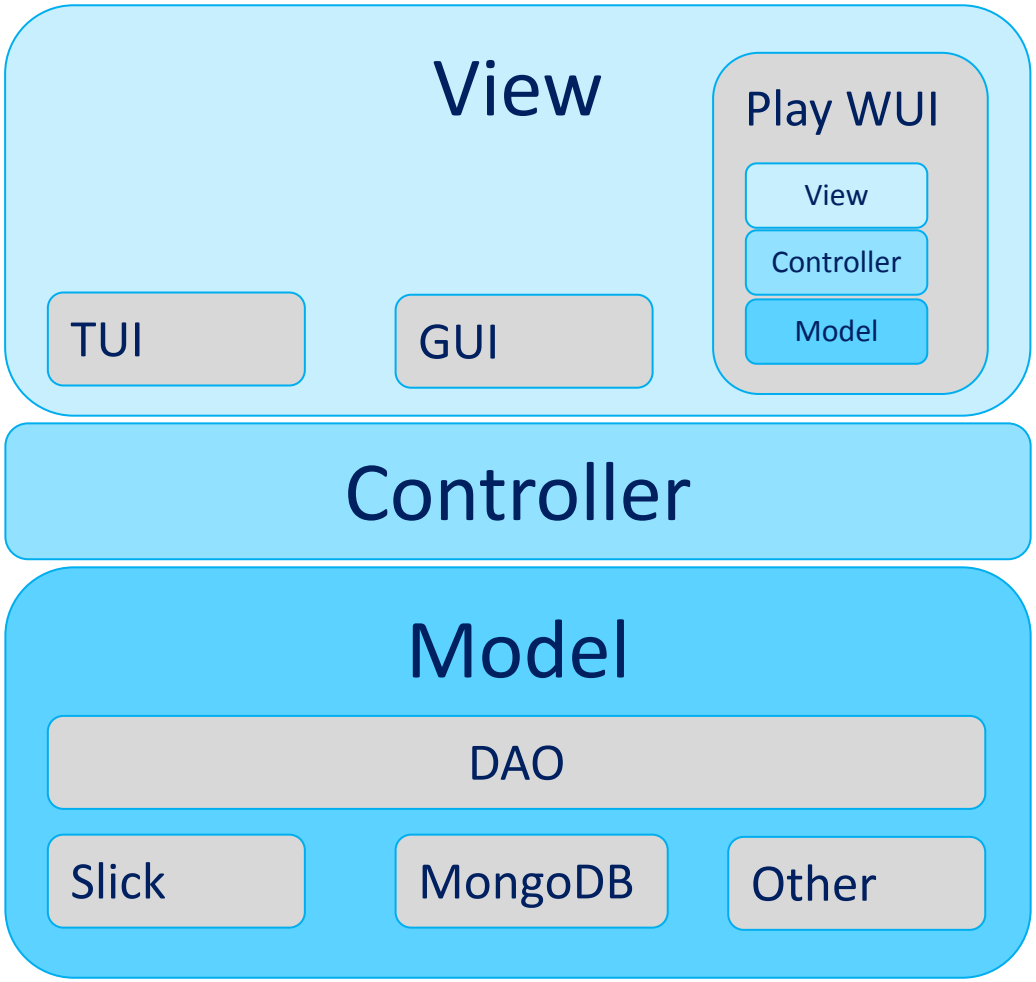


Architecture for this Lecture

- 3 different views
- Play has its own internal MVC

- 3 strickt layers

- 2 different DBs with a data access layer

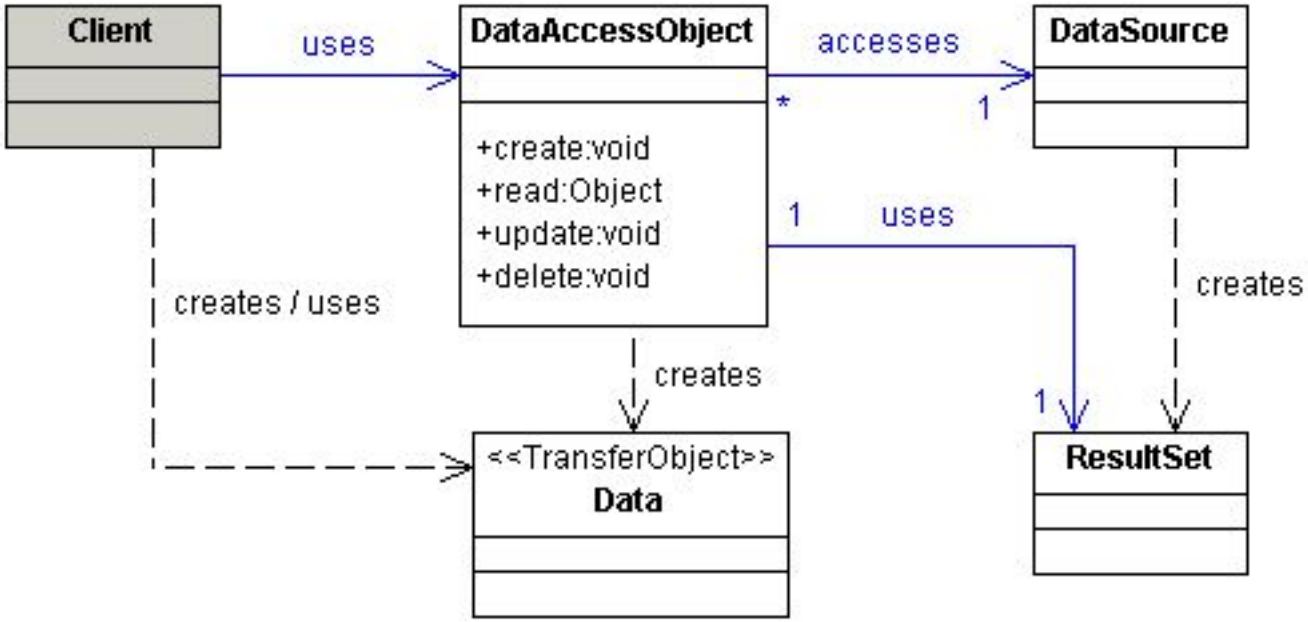


DAO: Data Access Object

- A design pattern
  - originally promoted by Sun for JavaEE
  - Equally applicable in other languages and databases
- A DAO provides an abstract interface to a persistence mechanism.
- The DAO hides the details of the database technology
- And make the database interchangeable
- The implementation of the DAO interface

DAO Pattern

- The Client uses a DAO to store and retrieve Data to a DB
- The DAO
  - opens, manages and closes DB connections
  - Transforms Data to the form expected by the client



Scala and JDBC

- Works, but is as verbose as in Java
- Doesn‘t match the functional paradigm at all

```
def personsMatching(pattern: String)(conn: Connection) = {
  val st = conn.prepareStatement(
    "select id, name from person where name like ?")
  try {
    st.setString(1, pattern)
    val rs = st.executeQuery()
    try {
      val b = new ListBuffer[(Int, String)]
      while(rs.next)
        b.append((rs.getInt(1), rs.getString(2)))
      b.toList
    } finally rs.close()
  } finally st.close()
}
```

Objects vs. Relations

- OO Languages and Relational Tables do not match very well
  - Granularity
  - Inheritance
  - Identity
  - Association
  - Data navigation
- Querying data in SQL requires a transformation of the data into objects
- Storing objects also requires a transformation of the data
- This mapping can be automated to quite a high degree

Object-Relational Mapping

- This transformation is called Object-Relational Mapping
- Products and Standards have evolved since End of 90‘s
  - Toplink
    - Developed by The Object People, in 90‘s
    - Acquired by BEA, acquired by Oracle
    - In 2007 donated as open source, now EclipseLink
  - Hibernate
    - Started in 2001 by Gavin King
    - Created an alternative to EJB 2
    - Strongly influenced the development of EJB 3
    - Now maintained by Red Hat
  - Other ORMs: Cayenne, Kodo, Torque, iBatis, JPOX
  - EBean

Hibernate

- Translation of Hibernate is “Winterschlaf”
- Hibernate is a persistence Framework
  - Object-Relational Mapping
    - From Objects to Relational Databases (RDBMS)
    - Makes a RDBMS feel almost like an OODBMS
    - But allows to connect to existing RDBMS
    - Or use the scalability of RDBMS
  - Independant of the used RDBMS
    - Can map to almost any RDBMS
  - Highly configurable
  - JPA compatible
  - Open-Source

Java Persistence API

- A Standard developed by the Java Community Process
  - JCP develops Java Specification Requests (JSR)
    - JSR 220 – EJB 3, separated out JSR 317 – Java Persistence API
  - Strongly influenced by TopLink and Hibernate
    - Gavin King represented Jboss in the JSR
    - TopLink and Hibernate both conform to JPA
  - Metadata defines the mapping from OO to Relations
    - Either as Annotations
    - Or as XML

Scala and existing (Java-)ORMs

- Works quite good with some limitations
  - Querys work with Java Collections instead of Scala collections
  - ORMs expect entities to be mutable
  - No support for Option types

```
@Entity
@Table(name = "person")
class Person(first: String, last: String) {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    var id: Int = _

    var firstName: String = first
    var lastName: String  = last

    def this() = this (null, null)

    override def toString = id + " = " + firstName + " " + lastName
}
```

Hibernate for Sudoku

- We need to store Grid and Cell
  - House and Status do not need to be persisted
- We could annotate Grid and Cell directly
  - But we want all changes for Hibernate in a separate package
- Instead we use PersistentGrid and PersistentCell
  - We will annotate these
  - Copy data from Grid to PersistentGrid, Cell to PersistentCell
  - And persist PersistentGrid and PersistentCell
  - For loading we will do the inverse
- That way, all Hibernate related stuff is isolated

Annotations for PersistentGrid

```
@Entity
@Table(name = "grid")
public class PersistentGrid implements Serializable {

@Id
@Column(name = "id")
@GeneratedValue(strategy = GenerationType.AUTO)
public Integer id;

@OneToMany(mappedBy = "grid")
@Column(name = "id")
public Set<PersistentCell> cells;

public  PersistentGrid() {
}
...
```

The O/R Impedance Mismatch

- The relational model is set-oriented
  - It consists of relations, tuples and attributes
  - Each tuple represents a fact, something that is true
  - Data retrieval via predicate logic
  - Uses set-operations like project, join, union or intersect
- In contrast, The object-oriented model is based on..
  - Identity and State
  - Behavior
  - Encapsulation

Paradigms don‘t match at all. OR-Mapping tools can close the gap only to a certain degree, but cannot resolve the mismatch entirely

Object-to-Table Mapping Problem

Basic approach: map tables to types, columns to attributes

- But what about Inheritance?
  - The relational model is not aware of IS-A or polymorphism semantics, common ORM strategies are a compromise
- What about associations?
  - unidirectional / bidirectional vs. database normalization
  - 1:N relationships are often reversed
  - N:M relationships need an additional join-table and joins

The Data Retrieval Concern

- How to retrieve data using an ORM?
- Most OR-Mappers provide 3 types of approaches
  - Query-by-Example (QBE)
  - Query-by-API (QBA)
  - Query-by-Language (QBL)

- Variety of approaches is necessary
  - Due to weaknesses in each approach
  - To complement each other

Query-by-Example (QBE)

- Uses a template / prototype of an object

```
Account account = new Account();
account.setName("Davis");
account.setBalance(5000);

Example accountExample = Example.create(account);
List<Account> result = accountExample.list();
```

- Good for simple queries, not expressive enough for more
- Negative impact on domain model restrictions
  - Attributes must be null at the beginning (or at least be nullable)

Query-by-Language

- The most expressive API, usually a subset of SQL
- But: it‘s verbose, not typesafe, no IDE support

```
String qbl = "FROM Account"
           + "WHERE name = :name "
           + "AND a.balance < :balance "
           + "ORDER BY name";

Query query = session.createQuery(qbl);
query.setParameter("name", "Ross").setParameter("balance", 0);
List list = query.list();
```

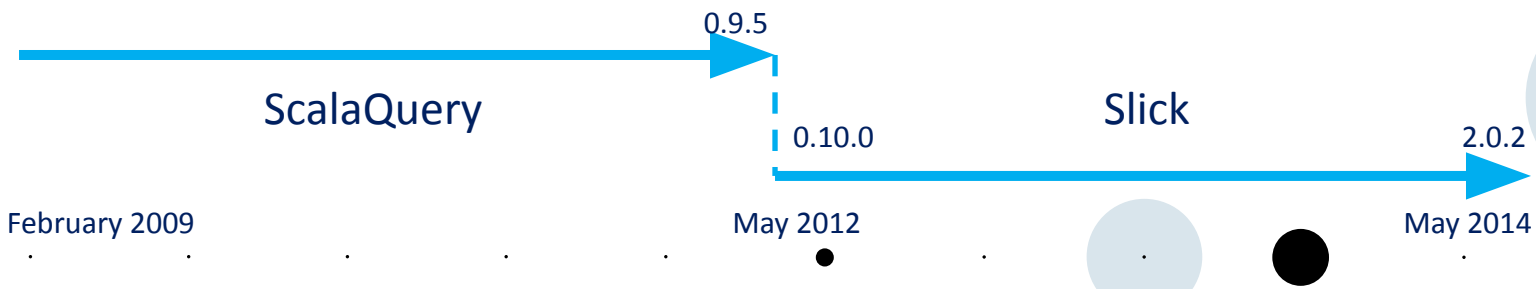
“Object/Relational Mapping is the Vietnam of Computer Science”

- Ted Neward

http://blogs.tedneward.com/2006/06/26/The+Vietnam+Of+Computer+Science.aspx

Slick

- Slick is a database access and query library for Scala
- Slick – Scala Language Integrated Connection Kit
- Is developed at Typesafe by Stefan Zeiger
  - Typesafe was founded by Martin Ordersky
  - also known for Akka, Play!, SBT, the Eclipse Scala IDE and Scala itself
- Is the successor of ScalaQuery, also developed by Stefan Zeiger



Query-by-API

- Often provides a fluent interface
- More verbose than SQL itself
- Is possibly not typesafe

```
Criteria crit = session.createCriteria(Account.class);
crit.add(Restrictions.eq("name", "Ross"))
    .add(Restrictions.lt("balance", 0))
    .addOrder(Order.asc("name"))
    .setMaxResults(10);

List<Account> result = crit.list();
```

A Functional-Relational Mapping

- Slick is a functional-relational mapper for Scala
- But Scala is an object-functional language
- Slick is not an ORM!

The idea behind Slick

- Slick embraces the relational model instead of abstracting from it
  - The database contains the concept of relations and so does the application

```
class Accounts extends Table[...]("accounts") { ...
}
```

- Write database code in Scala instead of JPQL, HQL etc…
  - Slick provides a typesafe Scala collection-like API
  - Scala programmers are already familiar with it

```
val accounts = TableQuery[Accounts]
val query = accounts.filter(_.id > 145)
```

Including Slick with SBT

```
libraryDependencies ++= Seq(
  "com.typesafe.slick" %% "slick" % "3.0.0",
  "org.slf4j" % "slf4j-nop" % "1.6.4"
)
```

Supported Database Systems

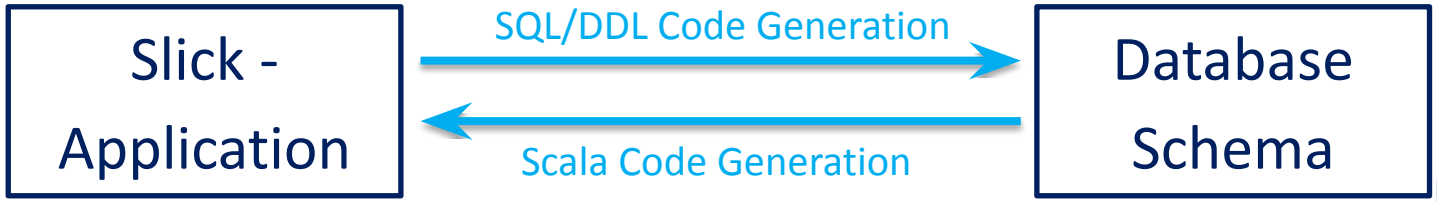
- Out-of-the-box
  - MySQL
  - PostgreSQL
  - Derby / JavaDB
  - H2
  - HSQLDB / HyperSQL
  - SQLite
  - Microsoft Access
  - Oracle *
  - Microsoft SQL Server *
  - DB2 *

* Requires a typesafe subscription and the slick-extensions package which contains closed source drivers

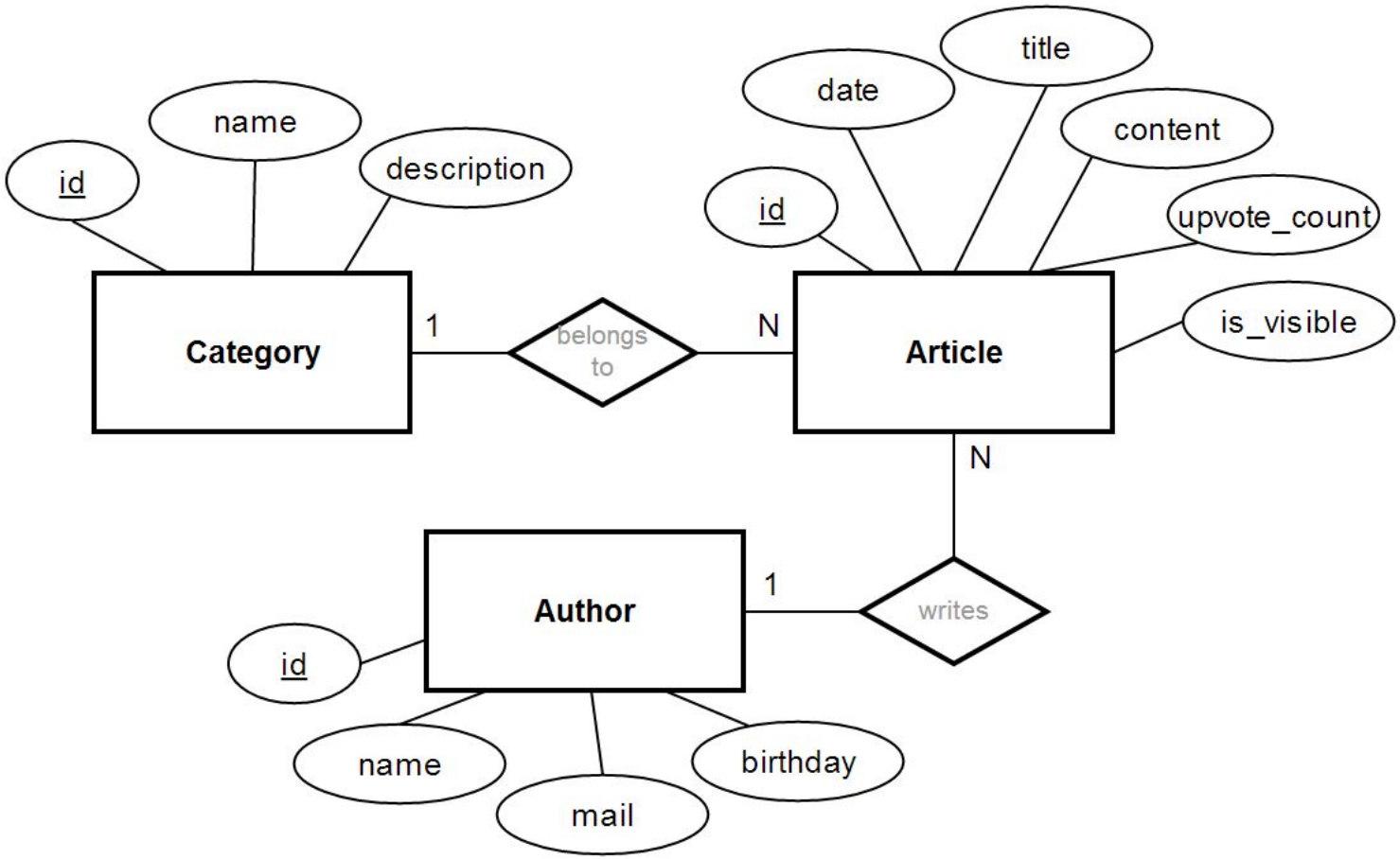
Database Schemas

Slick is mapped to a relational Database Schema.

- Mapping can be done in both directions.
  - Slick provides a Scala code generator for working with existing databases and database schemas (new feature of Slick 2.0)
  - Alternatively, a database schema can be defined on the application level using Scala which results in SQL DDL-statements



Sample App Data Model



Defining a Schema

Schema description for the category entity:

```
class Categories extends Table[(Int, String,
Option[String])]("categories") {

  // column definition
  def id = column[Int]("id", O.PrimaryKey, O.AutoInc)
  def name = column[String]("name")
  def description = column[Option[String]]("description")

  // default projection
  def * = (id, name, description)

}
```

Defining a Schema

Schema description for the author entity:

```
class Authors extends Table[(Int, String, Option[String],
Date)]("authors") {

  // column definition
  def id = column[Int]("id", O.PrimaryKey, O.AutoInc)
  def name = column[String]("name")
  def emailAddress= column[Option[String]]("email")
  def registrationDate = column[Date]("registration_date")

  // default projection
  def * = (id, name, emailAddress, registrationDate )

}
```

Defining a Schema

Schema description for the articles entity:

```
class Articles extends Table[Int, Int, Int, Date, String, String, Int,
Boolean)]("art") {

  // column definition
  def id = column[Int]("id", O.PrimaryKey, O.AutoInc)
  def catId = column[Int]("category_id")
  def authorId = column[Int]("author_id")
  def date = column[Date]("date")
  def title = column[String]("title"))
  def content = column[String]("content", O.DBType("CLOB"))
  def upvoteCount = column[Int]("upvote_count", O.Default(0))
  def isVisible = column[Boolean]("visible", O.Default(false))
  // default projection
  def * = (id, catId, authorId, date, title, content, upvoteCount, isVisible)
  // foreign key
  def categories = foreignKey("cat_fk", catId, TableQuery[Categories])(_.id)
  def authors= foreignKey(“auth_fk", authorId, TableQuery[Authors])(_.id)

}
```

Supported Data Types

- String
- Boolean
- Numeric Types
  - Byte, Short, Int, Long, BigDecimal, Float, Double
- Date types
  - java.sql.Date, java.sql.Time, java.sql.Timestamp
- LOB types
  - java.sql.Blob, java.sql.Clob, Array[Byte]
- java.util.UUID
- Unit

Deploying a Schema

Slick generates DDL statements for each Table

```
val articles = TableQuery[Articles]
articles.ddl.createStatements foreach println
```

```
CREATE TABLE "articles" (
    "id" INTEGER GENERATED BY DEFAULT NOT NULL PRIMARY KEY,
    "category_id" INTEGER NOT NULL,
    "author_id" INTEGER NOT NULL,
    "date" DATE NOT NULL,
    "content" CLOB NOT NULL,
    "upvote_count" INTEGER DEFAULT 0 NOT NULL,
    "visible" BOOLEAN DEFAULT false NOT NULL)
ALTER TABLE "articles"
    ADD constraint "cat_fk" FOREIGN KEY("category_id") REFERENCES
"categories"("id")
ALTER TABLE "articles"
    ADD constraint "auth_fk" FOREIGN KEY("author_id") REFERENCES "authors"("id")
```

Deploying a Schema

- Possible concatenation of ddl objects
- Slick takes care of (cyclic) dependencies
- Automatic deployment with create / drop

```
val ddl = (categories.ddl ++ articles.ddl ++ authors.ddl)

// -> within session…

    ddl.create
    // …
    ddl.drop
```

Database Connection

Slick provides several factory methods for establishing database connections

- Via JDBC-URL

```
val db = Database.forURL("jdbc:h2:mem:test1", driver = "org.h2.Driver")
```

- Via DataSource

```
val db = Database.forDataSource(dataSource)
```

- Via JNDI lookup

```
val db = Database.forName("JNDI-name")
```

Session handling

- Auto-closing session scope

```
val query = articles.filter(_.upvoteCount > 50)
val result = db.withSession {
    session => query.list()( session )
}
```

- Implicit session

```
val query = articles.filter(_.upvoteCount > 50)
val result = db.withSession {
    session => query.list()( session )
}
```

Inserts

- Insertion of a single element
- Expects all columns from the table‘s default projection

```
authors += (0, "Jane Doe", Some("jane@mail.com"), Date.valueOf("1965-05-18"))
authors += (1, "York Warden", Some("york@warden.com"), Date.valueOf("1979-12-05"))
authors += (2, "Arlo Darnell", None, Date.valueOf("1971-03-22"))
```

```
INSERT INTO "authors" ("name","email","birthday") VALUES (?,?,?)
INSERT INTO "authors" ("name","email","birthday") VALUES (?,?,?)
INSERT INTO "authors" ("name","email","birthday") VALUES (?,?,?)
```

Inserts with Omission

- Omitting columns using map

```
// omitting id & email
val insert = authors.map(a => (a.name, a.birthday))

insert += ("Jane Doe", Date.valueOf("1965-05-18"))
insert += ("York Warden", Date.valueOf("1979-12-05"))
insert += ("Arlo Darnell", Date.valueOf("1971-03-22"))
```

Queries

Queries follows a 2-step pattern

1. Select data, independent of operation to apply

```
val articles = TableQuery[Articles]
val query = articles
        .filter(_.upvoteCount > 50)
            .filter(_.title like "%Scala%")
        .sortBy(_.date)
        .drop(10).take(5)
        .map(_.title)
```

2. Apply an operation (triggers execution)

```
val result = query.list

// or: query.update(val: T), query.delete …
```

Queries with map or for

Queries can be expressed in two ways

- Higher order functions

```
val q1 = articles.filter(_.title like "%Scala%").map(_.id)
```

- For expressions

```
val q2 = for {
    a <- articles if a.name like "%Scala%"
} yield (a.id)
```

→ `SELECT id FROM articles WHERE title LIKE '%Scala%'`

Joins

```
val innerJoinA = for {
    a <- articles
    c <- categories if a.catId === c.id
} yield (c.name, a.title)
```

```
SELECT c.name, a.title
FROM categories c, articles a
WHERE c.id = a.cat_id
```

Updating and Deleting

- Deleting and updating rows is similar to selecting rows
- Invocation of delete/update instead of list

```
val q1 = authors.filter(_.id === 132).map(_.name)
val affectedRows = q1.update("York Wardell")

UPDATE authors SET name = 'York Wardell' WHERE id = 132

val q2 = authors.filter(_.id === 132)
val affectedRows = q2.delete

DELETE FROM authors WHERE id = 132
```

Summary of Slick Features

- Scala collection-like API for querying the database
- SQL generated by Slick is predictable
- Slick provides type-safety to a high degree
- No or at least small configuration, done in Scala code
- Full control over query execution
- Re-usable / composable queries for flexibility
- Possible "fallback" to plain SQL if needed

Task 6

Develop a database layer.

Use the DAO Pattern to make the interface independent of the used DB.

Use Slick as first DB implementation.